锁的膨胀过程

关于偏向锁

预备知识CAS

什么是CAS呢？compare and swap 比较和交换，在intel的CPU中，使用cmpxchg指令实现；在Java发展初期，java语言是不能够利用硬件提供的这些便利来提升系统的性能的。而随着java不断的发展,Java本地方法(JNI)的出现，使得java程序越过JVM直接调用本地方法提供了一种便捷的方式，因而java在并发的手段上也多了起来；

CAS 操作包含三个操作数 —— 内存位置（V）、预期原值（A）和新值(B)。 如果内存位置的值与预期原值相匹配，那么处理器会自动将该位置值更新为新值 。否则，处理器不做任何操作。无论哪种情况，它都会在 CAS 指令之前返回该 位置的值；整个过程都是不可打断的，所以CAS是一个原子操作；主要是利用CPU的CAS指令，同时借助JNI来完成Java的非阻塞算法；

CAS的练习

假设有如下接口

```
package com.enjoy.cas;

public interface Account {
    //查询
    Integer query();

    //获取
    void acquire(Integer i);
}
```

有api针对这个Account接口的实现类的query和acquire调用

```
package com.enjoy.cas;

import java.util.ArrayList;
import java.util.List;

/**
 * zl
 */
public class AccountTest {

    public static void main(String[] args) throws InterruptedException {
        //test(new AccountNomal(10000));
        //test(new AccountSync(10000));
        test(new AccountCas(10000));
    }

    /**
     * 传入一个具体的Account来测试
     * @param account
     */
```

```
    static void test(Account account) throws InterruptedException {
        List<Thread> ts = new ArrayList<>();
        //定义500个线程 每个线程针对account取20
        for (int i = 0; i < 500; i++) {
            ts.add(new Thread(() -> {
                account.acquire(20);
            }));
        }
        //启动
        for (Thread t : ts) {
            t.start();
        }
        //等待他们全部指向完
        for (Thread t : ts) {
            t.join();
        }
        //查询还剩多少
        System.out.println(account.query());
    }
}
```

如何保证acqurie的操作是线程安全呢?

线程不安全的实现

```
package com.enjoy.cas;

/**
 * 没有锁没有CAS
 *
 */
public class AccountNomal implements Account {
    public AccountNomal(int balance){
        this.balance=balance;
    }
    private Integer balance;
    @Override
    public Integer query() {
        return this.balance;
    }

    @Override
    public void acquire(Integer i) {
        this.balance -= i;
    }
}
```

运行结果：大于或者等于0;

线程安全的加锁实现

```
package com.enjoy.cas;
```

```java
/**
 * sync
 * result :0
 *
 */
public class AccountSync implements Account {
    public AccountSync(int balance){
        this.balance=balance;
    }
    private Integer balance;
    @Override
    public Integer query() {
        return this.balance;
    }

    @Override
    public void acquire(Integer i) {
        synchronized (this) {
            this.balance -= i;
        }
    }
}
```

运行结果：始终未0

线程安全的无锁实现

```java
package com.enjoy.cas;

import java.util.concurrent.atomic.AtomicInteger;

/**
 * cas
 * result :0
 *
 */
public class AccountCas implements Account {
    public AccountCas(int balance){
        this.balance=new AtomicInteger(balance);
    }
    private AtomicInteger balance;
    @Override
    public Integer query() {
        return this.balance.get();
    }

    @Override
    public void acquire(Integer i) {

            while(true) {
            int prev = balance.get();
            int next = prev - i;
            if(balance.compareAndSet(prev, next)) {
                break;
            }
        }
```

```
    }
}
```

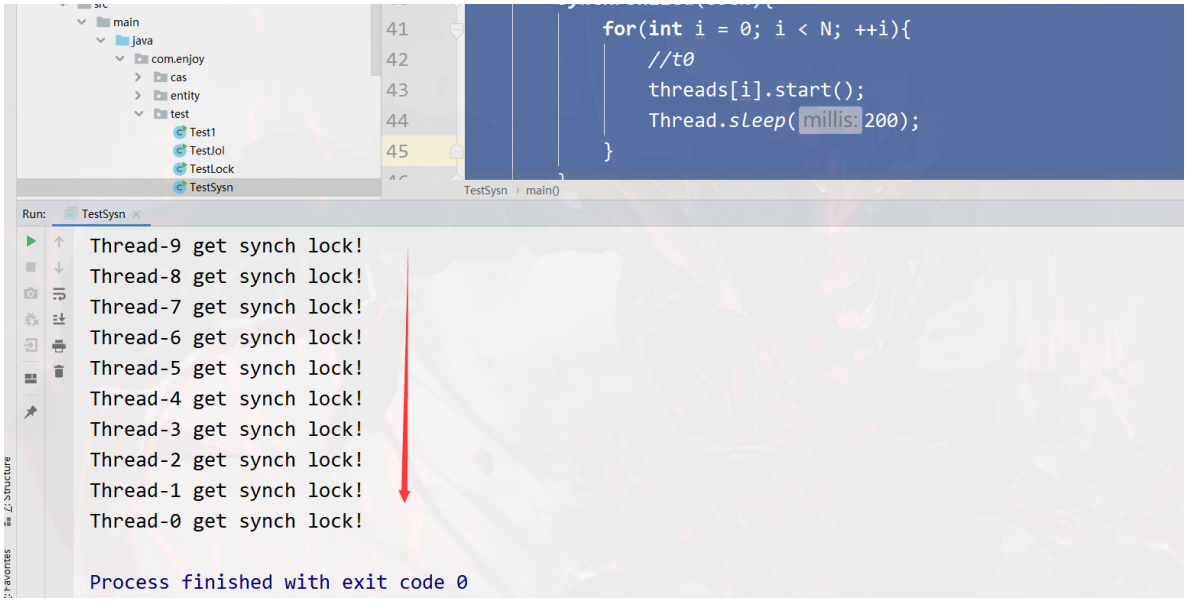
预备知识 公平锁非公平锁

考虑如下代码:

```
package com.enjoy.test;

/**
 * sync是否为公平锁
 */
public class TestSysn {
    //定义一把锁
    private static Object lock = new Object();

    public static void main(String[] args) throws InterruptedException {
        //线程的数量
        int N = 10;
        Thread[] threads = new Thread[N];


        for(int i = 0; i < N; ++i){
            threads[i] = new Thread(new Runnable(){
                public void run() {
                    /**
                     * 如果这里打印的结果是无序的则表示 非公平锁
                     * 有序则公平锁
                     * 结果是倒序 那么他就是公平锁吗?
                     * 其实很多时候可能我们对公平和非公平都理解错了
                     * 这里不管顺序打印还是倒序打印都只是对队列的特性验证
                     * 并不能说明他就是公平或者非公平
                     */
                    synchronized(lock){//t0 t1....t9 到一个队列当中的去阻塞
                        System.out.println(Thread.currentThread().getName() + "
get synch lock!");
                        try {
                            Thread.sleep(200);
                        } catch (InterruptedException e) {

                            e.printStackTrace();
                        }
                    }
                }

            });
        }

        //main 线程可以得到锁 持有了锁
        synchronized(lock){
            for(int i = 0; i < N; ++i){
                //t0
                threads[i].start();
                Thread.sleep(200);
            }
        }
```

```
//
//        for(int i = 0; i < N; ++i)
//            threads[i].join();


    }
}
```

结果分析：synchronized实现的锁，如果有多个线程阻塞，最先阻塞的最后执行；最后阻塞的最先执行

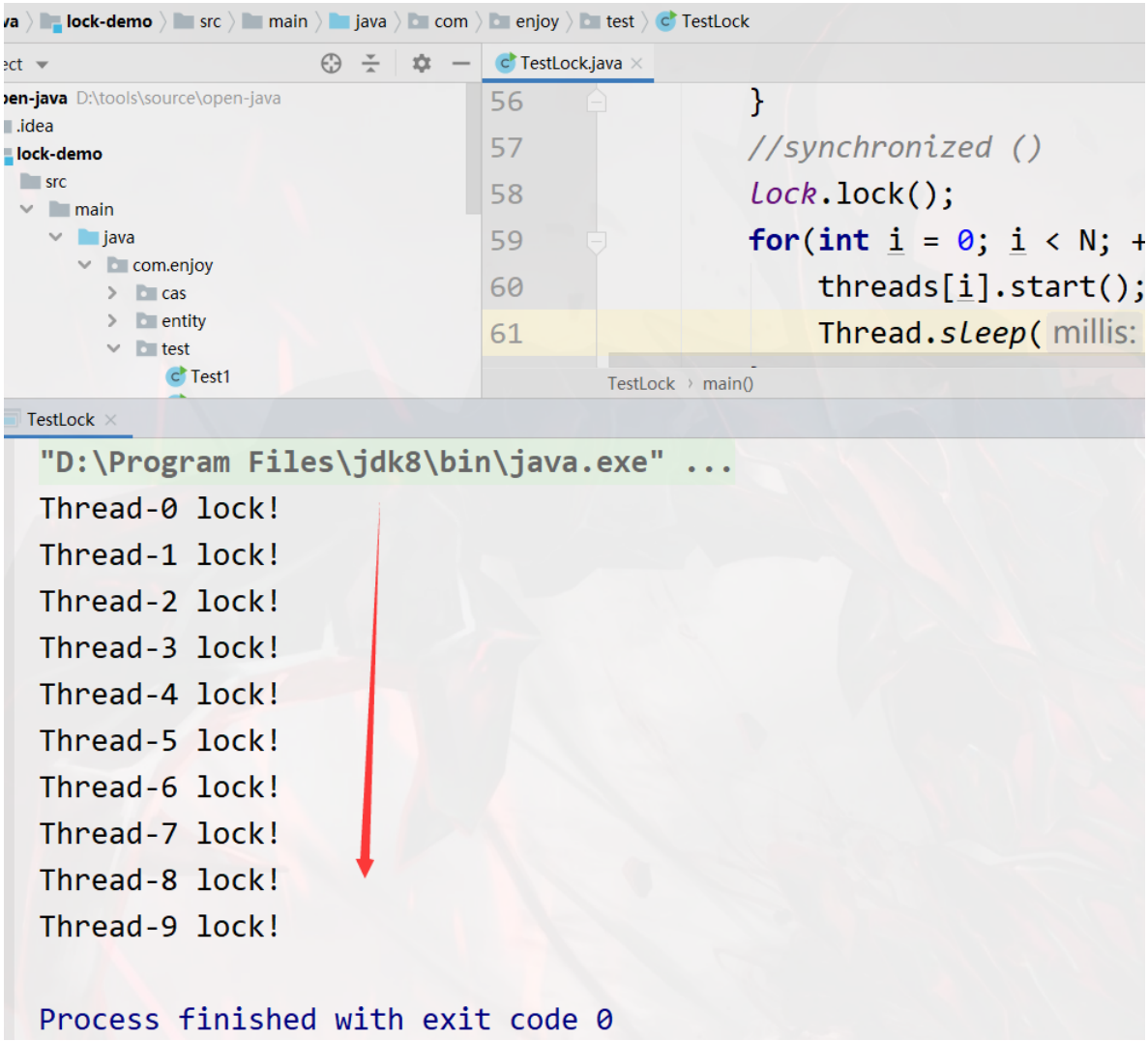


然后看看Lock实现的锁

```
package com.enjoy.test;

import java.util.concurrent.locks.Lock;
import java.util.concurrent.locks.ReentrantLock;

/**
 * 这是一把公平锁
 * 公平锁非公平锁理解本身就错了
 *
 *
 * 非公平锁他们首先会在lock方法调用加锁的时候去抢锁（公平锁调用lock不会上来就拿锁）；
 * 如果加锁失败
 * 则去看为什么失败（是否锁被人持有了），他在判断的时候如果锁没有被人持有
 * 非公平锁有会去直接加锁（不会判断是否有人排队），成功则进入同步块；失败则进入队列
 * 进入队列后如果前面那个人是head则再次尝试加锁，成功则执行同步代码块，失败则park（真正的排队）
 *
 * 公平锁：第一次加锁的时候，他不会去尝试加锁，他会去看一下我前面有没有人排队，如果有人排队，我则进入
 * 队列（并不等于排队），然后还不死心，再次看一下我有没有拿锁的资格（前面那个人是否为head），
 * 如果有资格（前面那个人刚好为head）则继续拿锁，成功则执行同步块；失败则park（排队）
 *
 *
 * 一朝排队，永远排队
 *
```

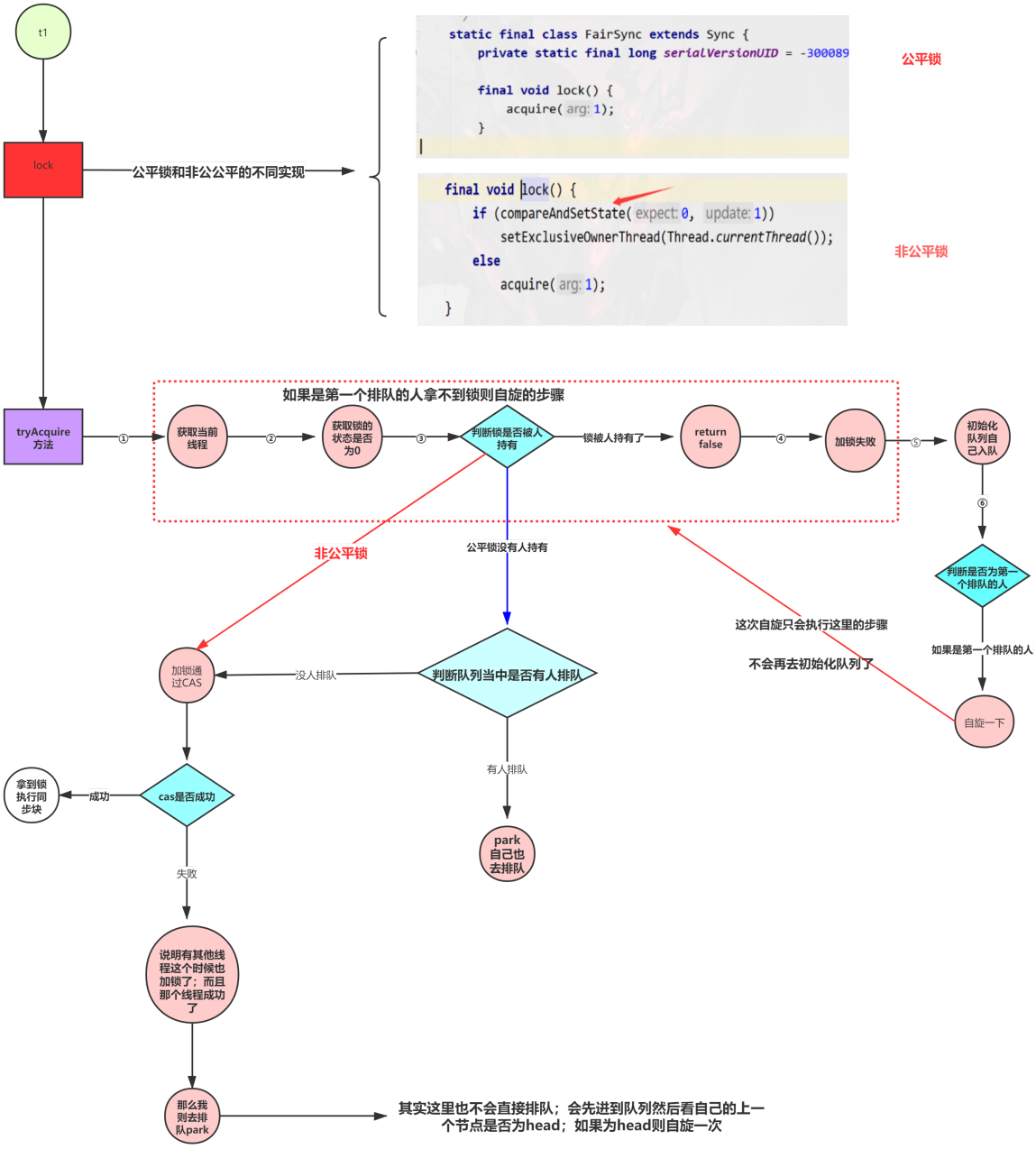
```
 */
public class TestLock {

    private static Lock lock = new ReentrantLock();

    public static void main(String[] args) throws InterruptedException {

        int N = 10;
        Thread[] threads = new Thread[N];
        for(int i = 0; i < N; ++i){
            threads[i] = new Thread(new Runnable(){
                public void run() {

                    /**
                     *
                     * 独占锁---顾名思义
                     * t1------t9全部在这里阻塞，并且进入到队列
                     * 等待持有锁的线程释放锁之后才从队列当中把这些阻塞的线程唤醒
                     *
                     */
                    lock.lock();
                    System.out.println(Thread.currentThread().getName() + "
lock!");
                    try {
                        Thread.sleep(20);
                    } catch (InterruptedException e) {
                        // TODO Auto-generated catch block
                        e.printStackTrace();
                    }
                    lock.unlock();
                }

            });
        }
        //synchronized ()
        lock.lock();
        for(int i = 0; i < N; ++i){
            threads[i].start();
            //Thread.sleep(200);
        }
       // lock.unlock();

        for(int i = 0; i < N; ++i)
            threads[i].join();
    }
}
```

结果分析：lock实现的同步如果有多个线程阻塞，唤醒的时候是按阻塞顺序唤醒的；关于公平锁和非公平锁可以参数我写的注释



关于公平锁和非公平锁的加锁流程和区别

好好看懂这幅图就懂了



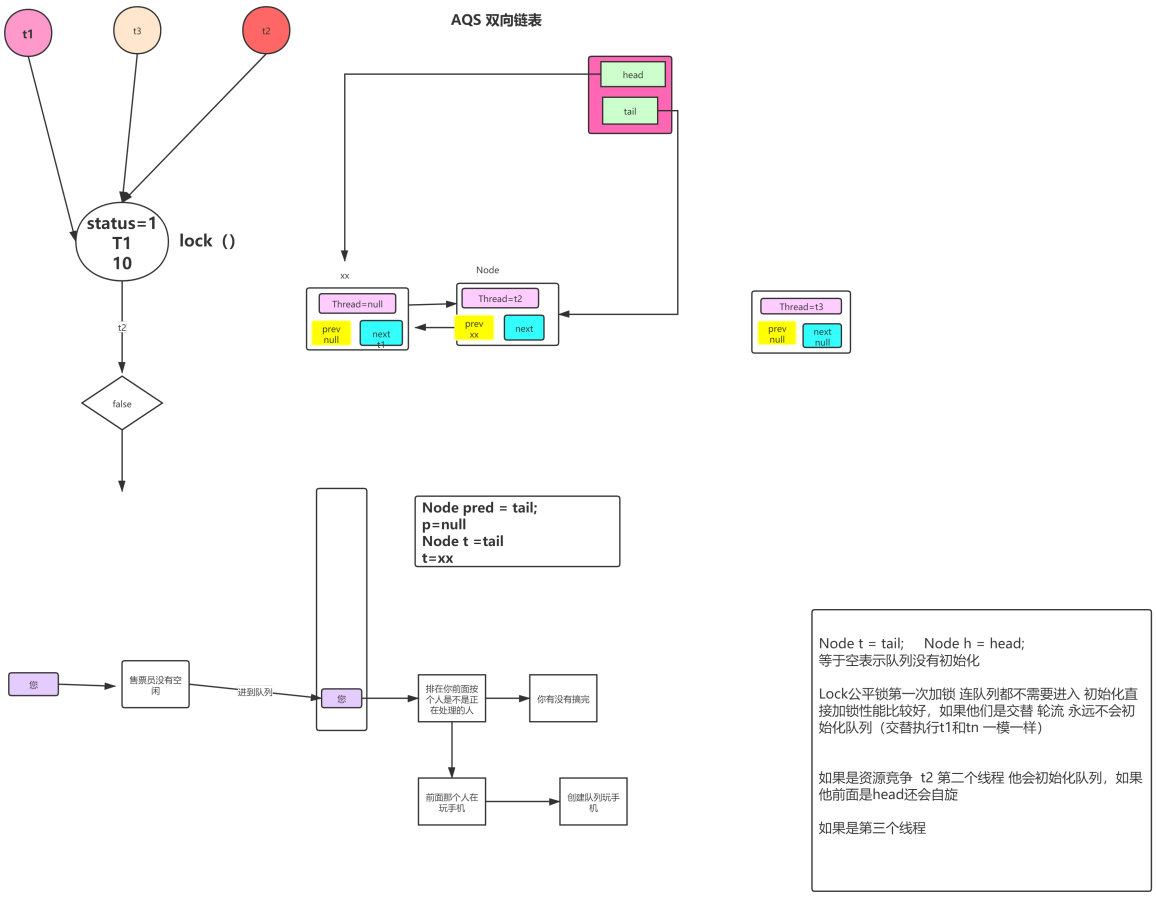
关于队列如何设计和形成

1、AQS类的设计主要属性

```
private transient volatile Node head; //队首

private transient volatile Node tail;//尾

private volatile int state;//锁状态，加锁成功则为1，重入+1 解锁则为0
```

2、Node类的设计

```
public class Node{
    volatile Node prev;
    volatile Node next;
    volatile Thread thread;
}
```



最后关于synchronized是否公平锁

其实前面那个小例子（倒序和顺序）不足以说明什么问题；synchronized是否公平锁得看线程在没有入队之前是否抢锁（C++代码）；入队之后就和公平不公平无关了；一朝排队；永远排队；

然后关于队列，关于双向链表好好理解一下就行

偏向锁膨胀过程

预备知识

1、什么叫做无锁；无锁分为两种情况 ①：无锁可偏向（101）②：无锁不可偏向（**0**01）在无锁不可偏向的情况下第一个0标识偏向标识不可偏向；但是还有一种情况也是101这种情况是有锁而且是已经偏向了线程；所以看一把锁（对象）是否有锁不能单纯的看后三位；比如后三位等于101；他可能是有锁；也有可能是无锁；但是后三位如果是001那么肯定是无锁；下文中说的无锁基本指的是001；

2、轻量锁：后两位是00(一共64位前面62位都是一个指针；所以轻量锁只需要看后两位00)；

3、比如一个l（在没有加锁的情况下；偏向没有延迟的情况下）new出来的时候是无锁可偏向也就是后三位是101；对象头的结构如下图；其中bl=1；lock=01

4、假设现在的场景是t1线程来加锁synchronized那么他必然是一个偏向锁；后三位同样还是101；但是前面的值改了；存了线程id和epoch等等信息；如下图；其中的bl=1；lock=01（注意和3不同的是前面的东西改了；后三位还是101）

5、然后t1把锁释放了（现在的场景是只有t1来加锁）；由于锁是偏向锁；故而就算释放了还是101；对象头还是和4一摸一；

6、总结一下；这种加锁模式我们课上没有说；仔细想想这种加锁模式是不是匿名偏向模式？就算线程来假设的时候发觉这把锁是匿名可偏向的；（因为来加锁的时候发觉是101；但是线程id没有；也就是不等于自己；然后再去判断是否匿名；这里成立；但是我们没有说匿名可偏向的加锁模式；下节课说）；

场景1、第二次偏向加锁

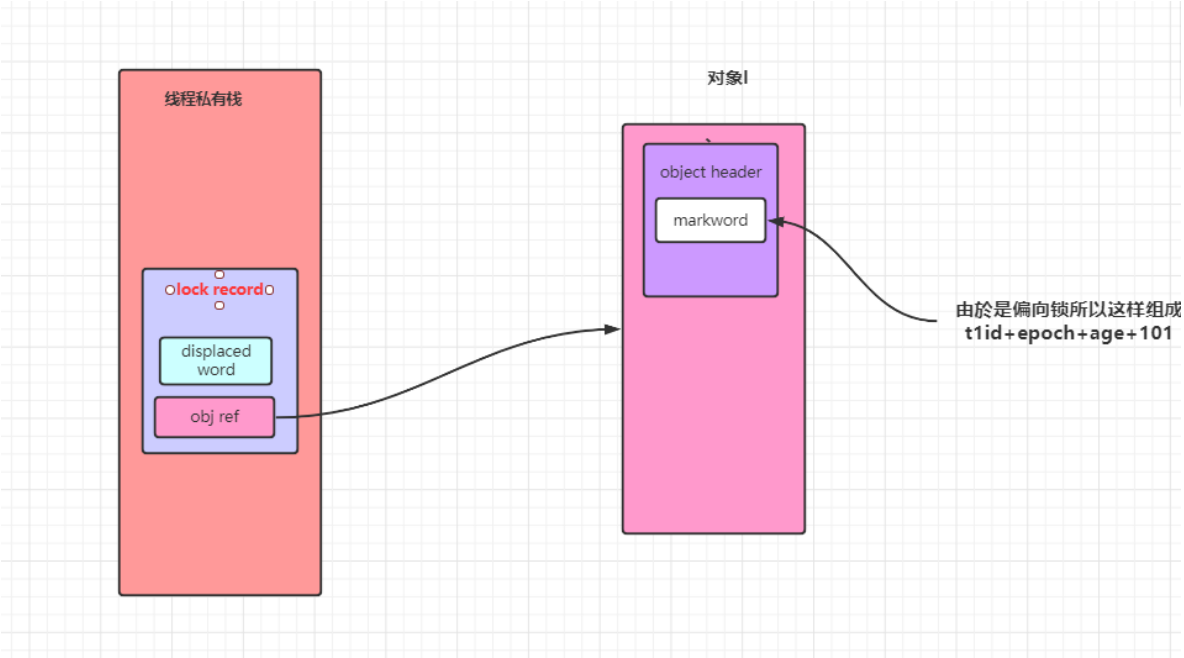
7、现在来考虑一下我们课上说的第一种情况；就是t1把锁释放之后；t1又来了加锁（由于是t1自己再次加锁所以还是偏向说；注意再次重新加锁和重入不一样）；

8、考虑重新加锁和重入的区别；其实很简单重新加锁是释放之后再次加锁；重入是没有释放之后再次加锁

```
sync(l){//1
    xxxxxx
}

sync(l){//2
    zzzzz
}
//上面的叫做重新加锁；如果是1的时候是偏向；那么2也是偏向


sync(l){//1
    xxxx
    sync(l){//2
        zzzz
    }
}

上面的代码叫做重入
```

9、回到7说的场景t1加锁；释放；然后再次加锁；我们课上说的就是偏向锁再次加锁的流程；

首先会有一个锁对象；比如l；然后会在当前线程栈当中创建一个lockrecord(以下简称lr)；



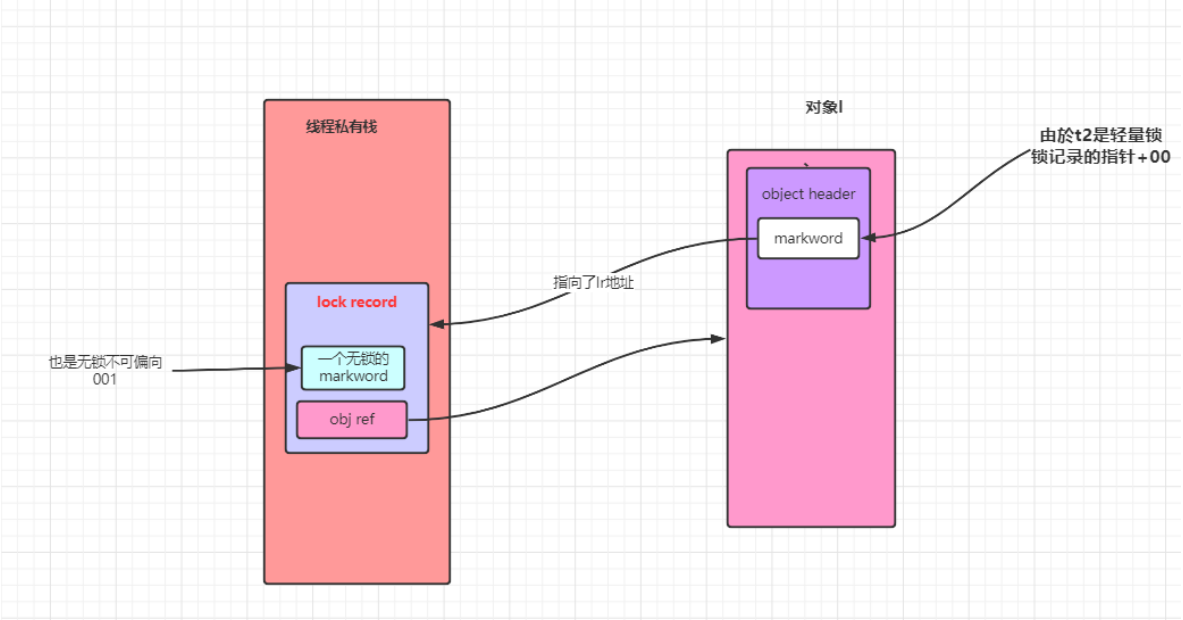
然后代码接着往下执行；判断当前线程是否自己；是否过期；不考虑过期那么线程直接拿到锁；这就是最简单的偏向锁获取过程

注意场景：是偏向锁第二次获取（包括后面第三次、第四次都是这样简单的判断一下）；

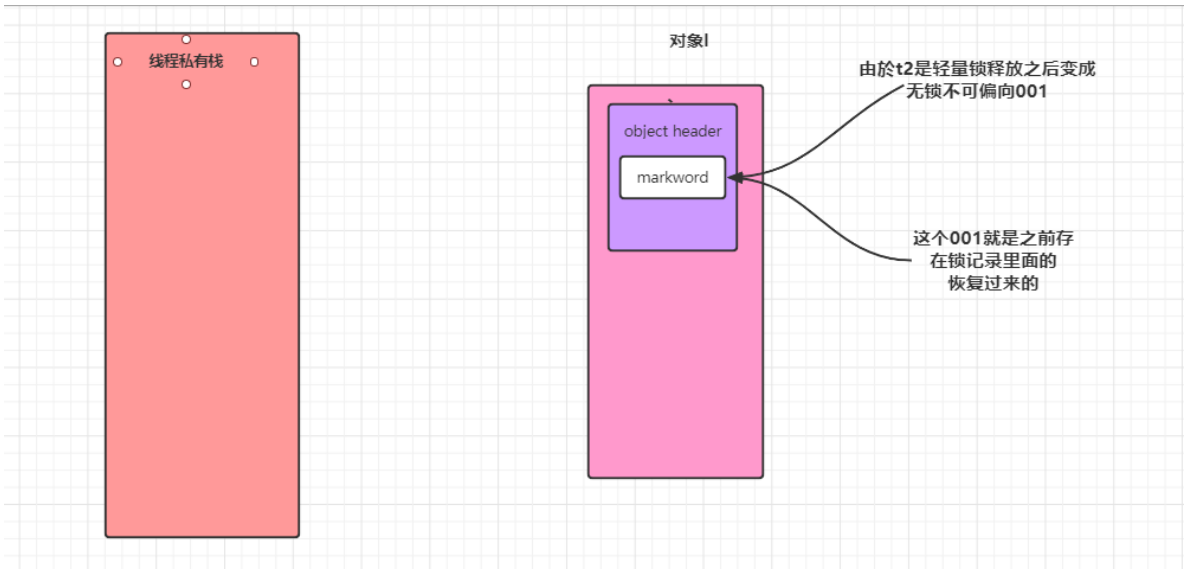
总结：偏向锁第二、三、四次（总之不是第一次、第一次没有讲）获取锁的流程：首选会在线程栈当中创建一个锁记录；这个锁记录主要的作用就是用来记录重入；创建完成后锁记录关系锁；然后代码接着往下执行；判断当前持有锁的的线程是否是自己；是否过期；如果满足则拿到锁；执行同步代码块；什么都不用改也不用CAS；性能最好；

10、假设第二种场景；现在t1线程执行完了；怎么释放的锁呢？就是从栈当中把那个锁记录lr给释放 所以偏向锁的锁释放也是很简单的（重入另外考虑）

11、又来假设一种场景就是现在t1释放锁了（偏向锁）；然后t2来了；根据我们的经验可知t2来加锁需要升级成为轻量锁；这里需要注意t2升级的时候需要做偏向撤销，我们没有讲（下节课讲）；总之t2如果加锁成功后内存变化如下图



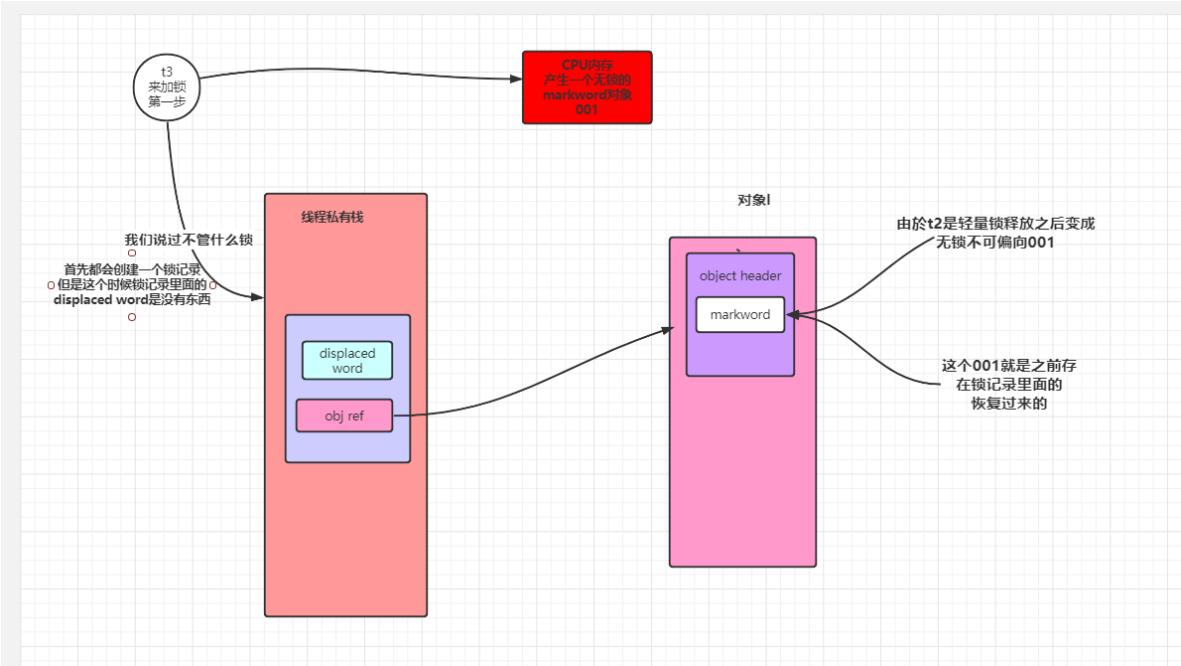
加锁t2执行完；我们已经得出结论；t2执行完后把锁记录释放并且恢复锁对象里面的markword；那么如果t2执行完同步块后的内存如下图



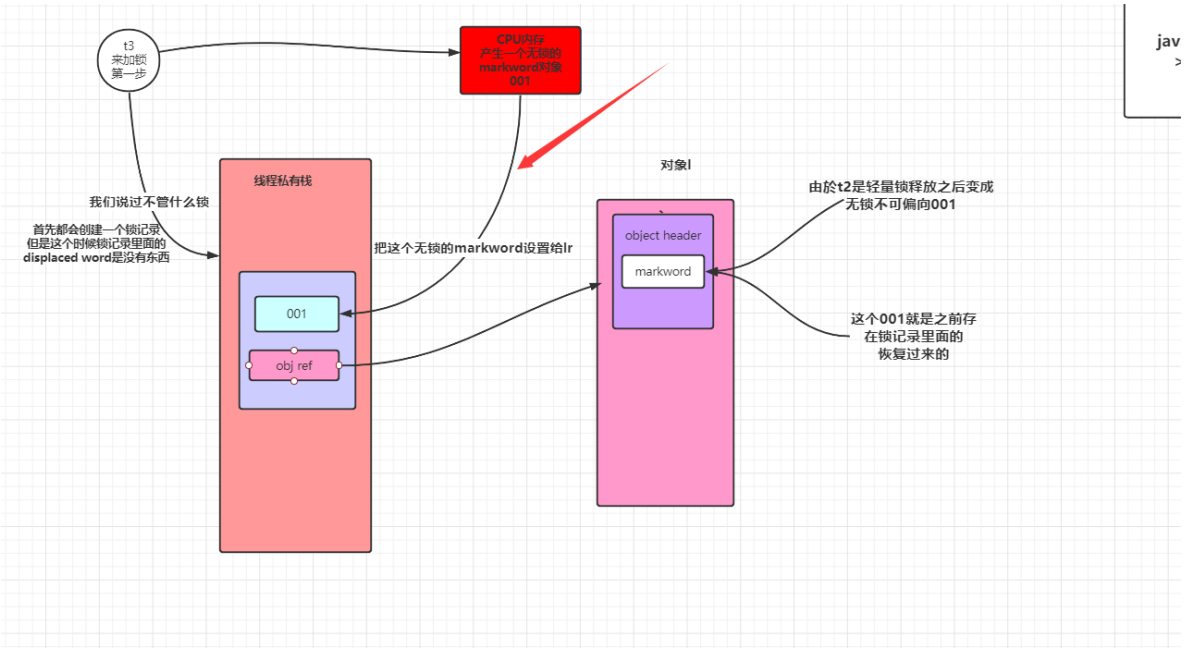
所以轻量锁的释放比较偏向锁的性能还是有区别的

场景2、轻量锁加锁（本来已经是轻量）

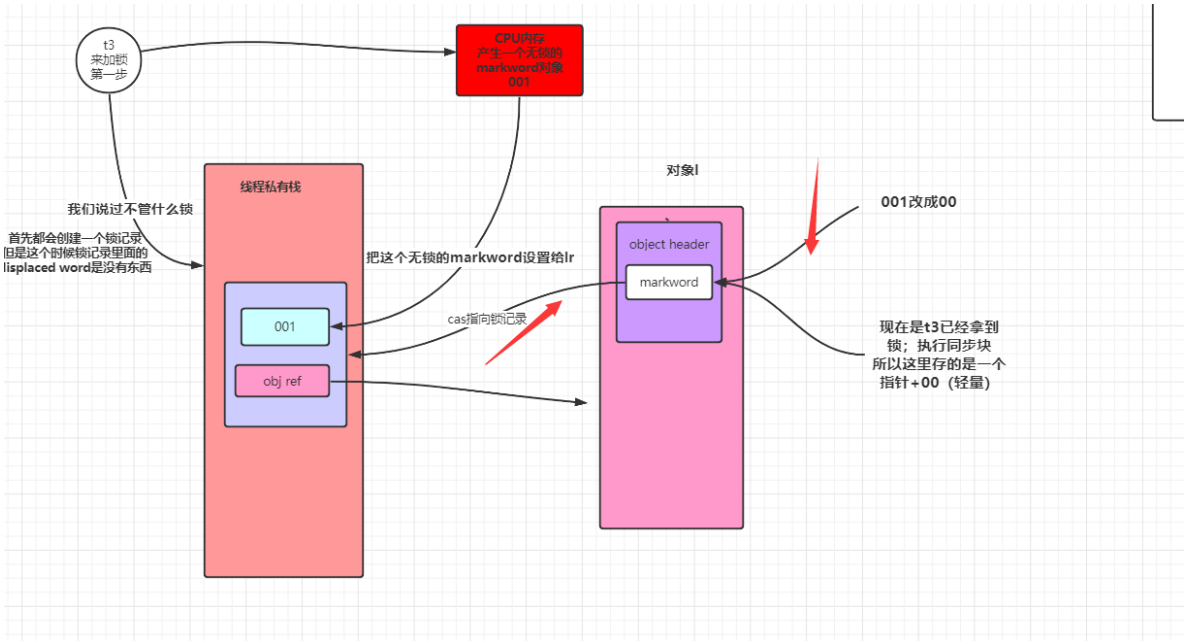
12、又来一种场景就是t2已经是轻量锁了；他把锁释放了；这个时候t3来了；那么t3首先生成一个无锁的markword 也就是001；内存如下图



然后把锁记录里面的displaced word设置成为这个无锁的红色箭头表示的；内存如下图



接着CAS 判断当前对象头当中的markword 是不是和t3在CPU内存当中产生的markword相等；如果相等 则吧对象头当中的markword修改成为一根指针指向锁记录；然后再把对象头当中的markword的后两位改成00；内存如下图

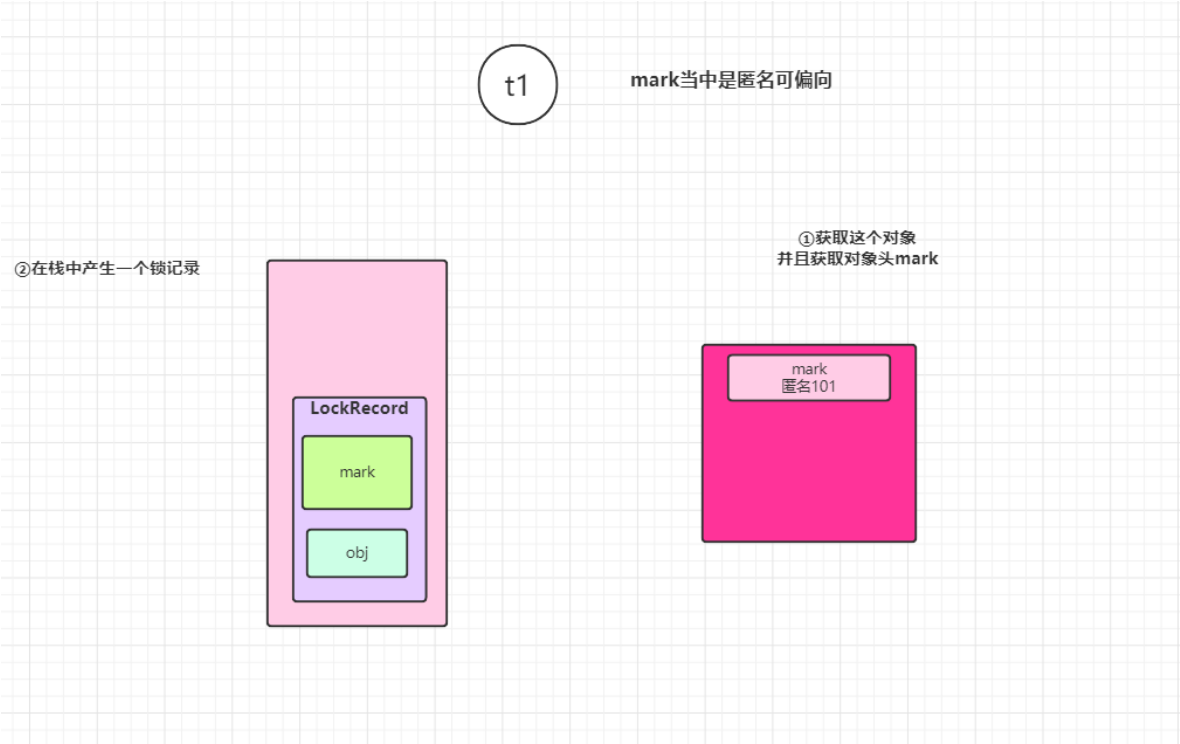


场景3、轻量膨胀重量

13 、那么什么时候会cas失败；就是另外一种场景了就是t4在t3还没有执行cas的时候他把锁持有了改成了00（把t3的事情做完了）；那么t3 cas失败；因为cas是 判断当前对象头当中的markword 是不是和t3在CPU内存当中产生的markword相等；由于t3产生的是一个无锁的001；而对象头现在被t4改成了轻量（000）；所以失败就会膨胀；

场景4、第一次加锁（第一次偏向）

14、再来一种场景：t1第一次加锁；直接偏向锁



会走匿名偏向锁的流程；产生一个偏向自己的mark；然后cas替换对象头；成功则加锁；失败则撤销偏向并且升级轻量；

场景5、批量重偏向

如果对象虽然被多个线程访问，但没有竞争，这时偏向了线程 T1 的对象仍有机会重新偏向 T2，重偏向会重置对象的 Thread ID；当撤销偏向锁阈值超过 20 次后，jvm 会这样觉得，我是不是偏向错了呢，于是会在给这些对象加锁时重新偏向至t2

代码测试批量重偏向

```
package com.enjoy.inflate;


import com.enjoy.entity.A;
import lombok.extern.slf4j.Slf4j;
import org.openjdk.jol.info.ClassLayout;

import java.util.ArrayList;
import java.util.List;
import java.util.concurrent.locks.LockSupport;

@Slf4j(topic = "enjoy")
public class TestInflate {

    static  Thread t2;
    static Thread t3;
    static Thread t1;
    static int loopFlag=38;
    public static void main(String[] args) throws InterruptedException {
```

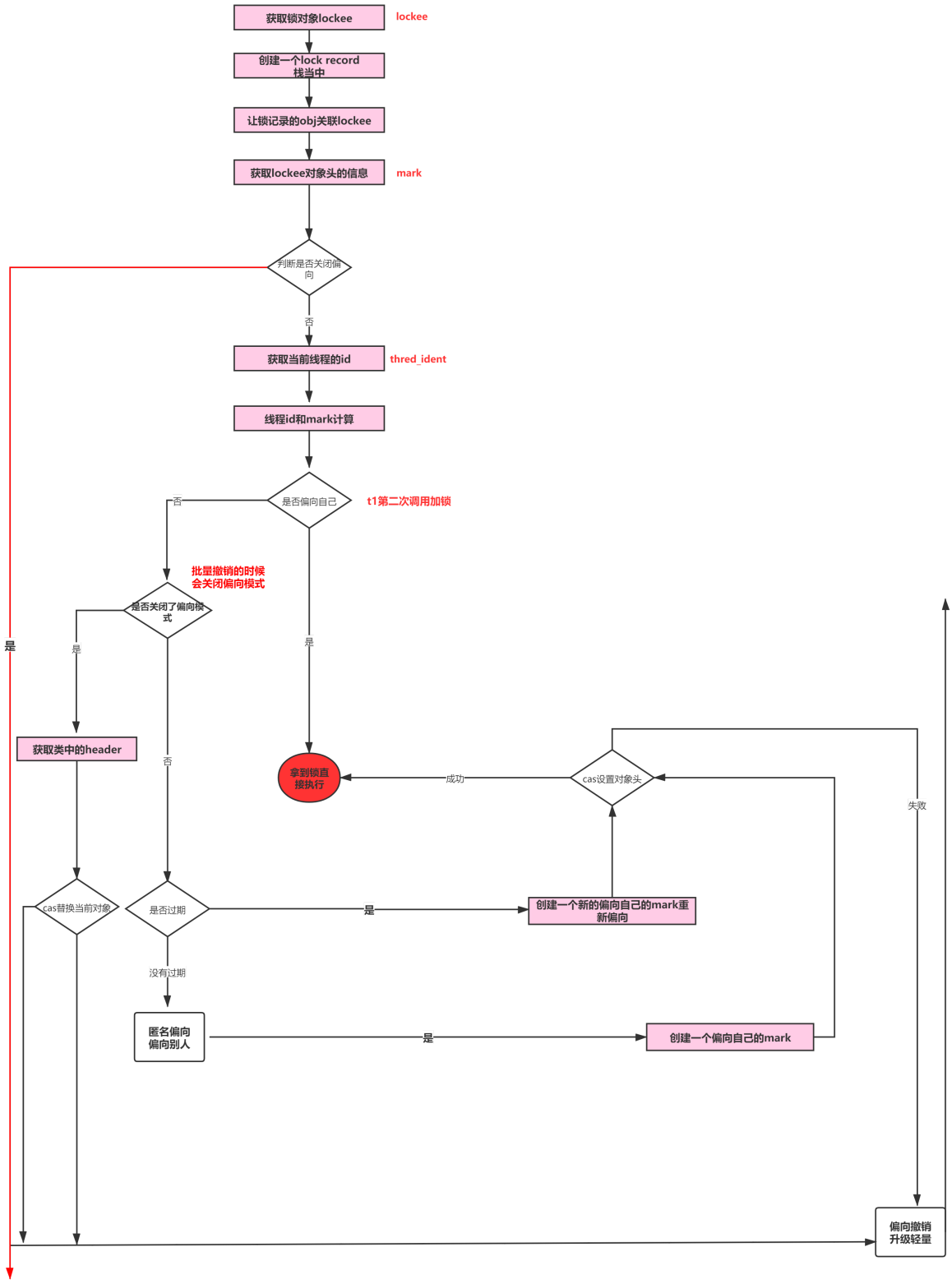
```
        List<A> list = new ArrayList<>();

        t1 = new Thread(){
           @Override
           public void run() {
               for(int i=0;i<loopFlag;i++){
                A a = new A();
                list.add(a);
                synchronized (a){
                    log.debug(i+"
"+ClassLayout.parseInstance(a).toPrintableTest(a));
                }
               }

               log.debug("============t1 都是偏向锁=============");
               //防止竞争 执行完后叫醒t2
               LockSupport.unpark(t2);
           }


       };

//
//
        t2 = new Thread(){

            @Override
            public void run() {
                //防止竞争 先睡眠t2
                LockSupport.park();
                    for (int i = 0; i <30 ; i++) {
                        A a = list.get(i);
                        //因为从list当中拿出都是偏向t1
                        log.debug("101首先before "+i+"
"+ClassLayout.parseInstance(a).toPrintableTest(a));
                        synchronized (a){
                            //前20撤销偏向t1；然后升级轻量指向t2线程栈当中的锁记录
                            //后面的发送批量偏向t2
                            log.debug("前20都是000 ing "+i+"
"+ClassLayout.parseInstance(a).toPrintableTest(a));
                        }
                        //因为前20是轻量，释放之后为无锁不可偏向
                        //但是后面的是偏向t2 释放之后依然是偏向t2
                        log.debug("前20都是001 after "+i+"
"+ClassLayout.parseInstance(a).toPrintableTest(a));
                    }
                log.debug("======t2 执行完后
newA=====================================");
                 //默认的A的对象应该是101
                log.debug("新产生的对象"+ClassLayout.parseInstance(new
A()).toPrintableTest(new A()));
                //log.debug("======t2=====================================");
                //LockSupport.unpark(t3);
            }
        };

        t1.start();
        t2.start();
```

```
    }


}
```

场景6、批量撤销

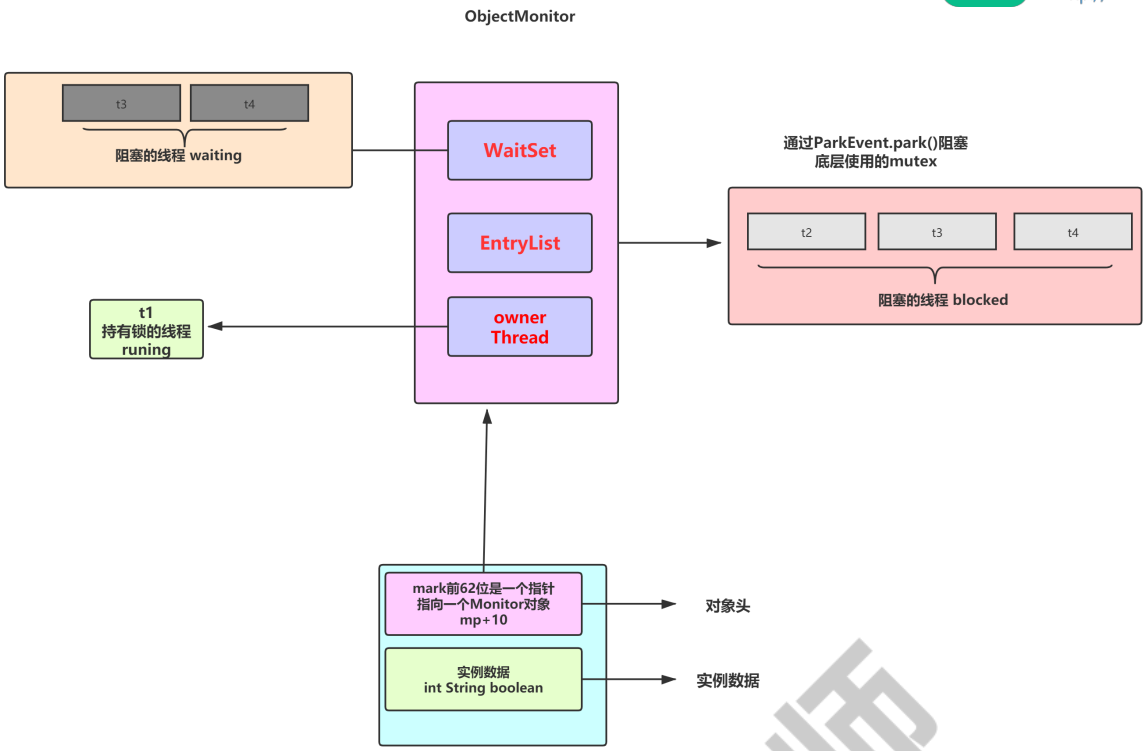
当一个偏向锁如果撤销次数到达40的时候就认为这个对象设计的有问题；那么JVM会把这个对象所对应的类所有的对象都撤销偏向锁；并且新实例化的对象也是不可偏向的

关于锁膨胀的流程图



synchronized 的工作模式 monitor

该文档是迅读PDF大师生成，
如果想去掉该提示，请访问并下载：
http://www.pdfxd.com



关于wait notify

原理参考上图

持有锁的线程发现条件不满足，调用 wait，即可进入 WaitSet 变为 WAITING 状态

BLOCKED 和 WAITING 的线程都处于阻塞状态，不占用 CPU 时间片

BLOCKED 线程会在 持有锁的线程释放锁时唤醒

WAITING 线程会在 持有锁的线程调用 notify 或 notifyAll 时唤醒，但唤醒后并不意味者立刻获得锁仍需进入EntryList 重新竞争

和sleep的区别

相同：线程的状态相同；都是阻塞状态

区别：

1、wait是Object的方法；任何对象都可以直接调用；sleep是Thread的静态方法

2、wait必须配合synchronized关键字一起使用；如果一个对象没有获取到锁直接调用wait会异常；sleep则不需要

2、wait可以通过notify主动唤醒；sleep只能通过打断主动叫醒

4、wait会释放锁、sleep在阻塞的阶段是不会释放锁的

深入理解wait 和notify sleep

考虑第一段代码

```
package com.enjoy.wait;
```

迅读PDF大师

该文档是迅读PDF大师生成，
如果想去掉该提示，请访问并下载：
http://www.pdfxd.com

```java
import lombok.extern.slf4j.Slf4j;

@Slf4j(topic = "enjoy")
public class TestWait1 {

    static final Object pc = new Object();
    static boolean isPrettyGril = false; // 有没有女人

    public static void main(String[] args) throws InterruptedException {
        new Thread(() -> {
            synchronized (pc) {
                log.debug("有没有女人[{}]", isPrettyGril);
                if (!isPrettyGril) {
                    log.debug("没有女人！等女人，等5秒;别人也不能干活");
                    try {
                        Thread.sleep(5000);
                    } catch (InterruptedException e) {
                        e.printStackTrace();
                    }
                }
                log.debug("等了5秒 有没有女人？[{}]", isPrettyGril);
                if (isPrettyGril) {
                    log.debug("------男女搭配干活不累；啪啪啪写完了代码");
                }else{
                    log.debug("------下班回家----");
                }
            }
        }, "jack").start();

        for (int i = 0; i < 10; i++) {
            new Thread(() -> {
                synchronized (pc) {
                    log.debug("我们10个屌丝工作了");
                }
            }, "其它人").start();
        }

       Thread.sleep(1000);
        new Thread(() -> {
            synchronized (pc) {
                isPrettyGril = true;
                log.debug("桥本有菜来了");
            }
        }, "boss").start();
    }




}
```

问题：sleep不会释放锁导致其他线程也不会执行

改进

```java
package com.enjoy.wait;
```

该文档是迅读PDF大师生成，
如果想去掉该提示，请访问并下载：
http://www.pdfxd.com

```java
import lombok.extern.slf4j.Slf4j;

@Slf4j(topic = "enjoy")
public class TestWait2 {

    static final Object pc = new Object();
    static boolean isPrettyGril = false; // 女人

    public static void main(String[] args) throws InterruptedException {
        new Thread(() -> {
            synchronized (pc) {
                log.debug("有没有女人[{}]", isPrettyGril);
                if (!isPrettyGril) {
                    log.debug("没有女人！等女人");
                    try {
                        pc.wait();
                    } catch (InterruptedException e) {
                        e.printStackTrace();
                    }
                }
                log.debug("老板喊我了 有没有女人？[{}]", isPrettyGril);
                if (isPrettyGril) {
                    log.debug("---男女搭配干活不累；啪啪啪写完了代码");
                }else{
                    log.debug("下班回家");
                }
            }
        }, "jack").start();

        for (int i = 0; i < 10; i++) {
            new Thread(() -> {
                synchronized (pc) {
                    log.debug("我们10个屌丝工作了");
                }
            }, "其它人").start();
        }

       Thread.sleep(1000);
        new Thread(() -> {
            synchronized (pc) {
                isPrettyGril = true;
                log.debug("桥本有菜来了");
                pc.notify();
            }
        }, "boss").start();
    }




}
```

问题：单个有条件的线程是没有问题的，设想如果有多个线程都需要满足条件

```java
package com.enjoy.wait;

import lombok.extern.slf4j.Slf4j;
```

```java
@slf4j(topic = "enjoy")
public class TestWait3 {

    static final Object pc = new Object();
    static boolean isPrettyGril = false; // 女人
    static boolean isMoney = false;//工资

    public static void main(String[] args) throws InterruptedException {
        new Thread(() -> {
            synchronized (pc) {
                log.debug("有没有女人[{}]", isPrettyGril);
                if (!isPrettyGril) {
                    log.debug("没有女人！等女人");
                    try {
                        pc.wait();
                    } catch (InterruptedException e) {
                        e.printStackTrace();
                    }
                }
                log.debug("老板喊我了 有没有女人？[{}]", isPrettyGril);
                if (isPrettyGril) {
                    log.debug("男女搭配干活不累；啪啪啪写完了代码");
                }else {
                    log.debug("不干了---下班回家--------");
                }
            }
        }, "jack").start();



        new Thread(() -> {
            synchronized (pc) {
                log.debug("有没有工资[{}]", isMoney);
                if (!isMoney) {
                    log.debug("没有工资！等发工资");
                    try {
                        pc.wait();
                    } catch (InterruptedException e) {
                        e.printStackTrace();
                    }
                }
                log.debug("老板喊我了 有没有发工资？[{}]", isMoney);
                if (isMoney) {
                    log.debug("-----卧槽好多钱；啪啪啪写完了代码");
                }else {
                    log.debug("约会去了");
                }
            }
        }, "rose").start();


       Thread.sleep(1000);
        new Thread(() -> {
            synchronized (pc) {
                isMoney = true;
                log.debug("发工资了---");
                pc.notify();
```

该文档是迅读PDF大师生成，
如果想去掉该提示，请访问并下载：
http://www.pdfxd.com

```
            }
        }, "boss").start();
    }




}
```

问题：虚假唤醒，怎么解决？

```
package com.enjoy.wait;

import lombok.extern.slf4j.Slf4j;

@Slf4j(topic = "enjoy")
public class TestWait4 {

    static final Object pc = new Object();
    static boolean isPrettyGril = false; // 女人
    static boolean isMoney = false;//工资

    public static void main(String[] args) throws InterruptedException {
        new Thread(() -> {
            synchronized (pc) {
                log.debug("有没有女人[{}]", isPrettyGril);
                if (!isPrettyGril) {
                    log.debug("没有女人！等女人");
                    try {
                        pc.wait();
                    } catch (InterruptedException e) {
                        e.printStackTrace();
                    }
                }
                log.debug("老板喊我了 有没有女人？[{}]", isPrettyGril);
                if (isPrettyGril) {
                    log.debug("男女搭配干活不累；啪啪啪写完了代码");
                }else {
                    log.debug("不干了---下班回家--------");
                }
            }
        }, "jack").start();



        new Thread(() -> {
            synchronized (pc) {
                log.debug("有没有工资[{}]", isMoney);
                if (!isMoney) {
                    log.debug("没有工资！等发工资");
                    try {
                        pc.wait();
                    } catch (InterruptedException e) {
                        e.printStackTrace();
                    }
```

```
                }
                log.debug("老板喊我了 有没有发工资？[{}]", isMoney);
                if (isMoney) {
                    log.debug("-----卧槽好多钱；啪啪啪写完了代码");
                }
            }
        }, "rose").start();

        for (int i = 0; i < 10; i++) {
            new Thread(() -> {
                synchronized (pc) {
                    log.debug("我们10个屌丝工作了");
                }
            }, "其它人").start();
        }

       Thread.sleep(1000);
        new Thread(() -> {
            synchronized (pc) {
                isMoney = true;
                log.debug("发工资了---");
                pc.notifyAll();
            }
        }, "boss").start();
    }




}
```

最终版

```
package com.enjoy.wait;

import lombok.extern.slf4j.Slf4j;

@Slf4j(topic = "enjoy")
public class TestWait5 {

    static final Object pc = new Object();
    static boolean isPrettyGril = false; // 女人
    static boolean isMoney = false;//工资

    public static void main(String[] args) throws InterruptedException {
        new Thread(() -> {
            synchronized (pc) {
                log.debug("有没有女人[{}]", isPrettyGril);
                while (!isPrettyGril) {
                    log.debug("没有女人！等女人");
                    try {
                        pc.wait();
                    } catch (InterruptedException e) {
                        e.printStackTrace();
                    }
```

```java
                }
                log.debug("老板喊我了 有没有女人？[{}]", isPrettyGril);
                if (isPrettyGril) {
                    log.debug("男女搭配干活不累；啪啪啪写完了代码");
                }else {
                    log.debug("不干了---下班回家--------");
                }
            }
        }, "jack").start();



        new Thread(() -> {
            synchronized (pc) {
                log.debug("有没有工资[{}]", isMoney);
                while (!isMoney) {
                    log.debug("没有工资！等发工资");
                    try {
                        pc.wait();
                    } catch (InterruptedException e) {
                        e.printStackTrace();
                    }
                }
                log.debug("老板喊我了 有没有发工资？[{}]", isMoney);
                if (isMoney) {
                    log.debug("-----卧槽好多钱；啪啪啪写完了代码");
                }
            }
        }, "rose").start();

        for (int i = 0; i < 10; i++) {
            new Thread(() -> {
                synchronized (pc) {
                    log.debug("我们10个屌丝工作了");
                }
            }, "其它人").start();
        }

       Thread.sleep(1000);
        new Thread(() -> {
            synchronized (pc) {
                isMoney = true;
                log.debug("发工资了---");
                pc.notifyAll();
            }
        }, "boss").start();
    }




}
```